

LA_pack primer

Abstract

GNU APL implements the APL functions monadic $\boxed{\div}$ (aka. **Matrix Inverse**) and dyadic $\boxed{\div}$ (aka. **Matrix Divide**) by means of *LApack functions*. These LApack functions were originally written in FORTRAN, but then manually translated from FORTRAN to C++ by the author. In this translation, features of the LApack functions that were not needed for $\boxed{\div}$ were removed in order to simplify the C++ code as much as possible.

Monadic $\boxed{\div}$: Matrix Inverse

Monadic $\boxed{\div}$ is the special case of dyadic $\boxed{\div}$ where A is the unit matrix:

Let $A \leftarrow (\uparrow pB) \circ. = (\uparrow pB)$. Then $\boxed{\div} B \leftrightarrow A p B$.

Bif_F12_DOMINO::eval_B(Value_P B) const, i.e. the implementation of monadic $\boxed{\div}$, simply constructs a unit matrix I of proper size and then returns **eval_AB(I, B)**, i.e. the result of the implementation of dyadic $\boxed{\div}$.

Dyadic $\boxed{\div}$: Matrix Divide

In APL, $\boxed{\div}$ for a N-column matrix A is defined in terms of each column of A :

If $X \leftarrow A \boxed{\div} B$ then $X[;J] \leftrightarrow A[;J] \boxed{\div} B$ for all (scalar) columns $J \in \iota^{-1} \uparrow pA$.

In other words, computing $\boxed{\div}$ for a matrix A boils down to computing $\boxed{\div}$ for (column-) vectors (which are the columns of A). For every such column vector a of A , the result is a column vector x of X :

$$B \circ x \equiv a \quad \leftrightarrow \quad x \equiv a \boxed{\div} B \quad \leftrightarrow \quad x \equiv a \circ B^{-1}$$



In GNU APL: $A +. \times B \leftrightarrow A \circ B$. The vast majority of inner products in real-life APL programs is used for matrix multiplication aka. $A +. \times B$. The GNU APL function $A \circ B$ is an optimized version of the special case $A +. \times B$ of the more general inner product $A f.g B$.

Since B is the same for all columns a of A (and their corresponding result columns x of X), is the result X of $X \leftarrow A \boxed{\div} B$ computed in two steps:

1. compute B^{-1}
2. compute $X \leftarrow A \circ B^{-1}$

The second step $X \leftarrow A \circ B^{-1}$ is simple, therefore only the first step, the computation of B^{-1} is of concern here.

BTW. the iteration over the columns of **A** in the second step above occurs near the end of function **LA_pack::gelsy()** in function **trsm<T>()**.

In order to compute **B⁻¹**, matrix **B** is factorized into two matrices **Q** and **R** which have the following properties:

- **Q** is orthogonal (resp. Hermitian for complex **B**). That is: **Q⁻¹ ≡ Q^H**,
- **R** is upper triangular, i.e. **R_{ij} = 0** for all **i > j**,
- **B = Q•R**,

Once **Q** and **R** are found, **B⁻¹** can be computed easily with:

$$\mathbf{B}^{-1} = (\mathbf{Q} \circ \mathbf{R})^{-1} = \mathbf{R}^{-1} \circ \mathbf{Q}^{-1} = \mathbf{R}^{-1} \circ \mathbf{Q}^H.$$

The inverse of an upper triangular matrix (such as **R⁻¹**) is far easier to compute than the inverse of a general matrix (such as **B⁻¹**). See below. What remains is therefore the computation of a QR factorization of an arbitrary matrix **B**.

Inversion of an orthogonal Q

The inversion of an orthogonal matrix **Q** means transposition of **Q** (and, in the complex case, conjugation of the complex items). However, unlike in APL where **QQ** is a new value computed from **Q**, this transposition is never computed in C++ or in FORTRAN. Instead, where **QQ** is needed, the items of matrix **Q** are simply accessed with the row and column interchanged. IOW, with a little bit of care as to what are rows and what are columns is **QQ** no-op. In GNU APL three typedefs are used to distinguish rows and columns throughout the code:

In **LApack.hh**:

```
typedef int Crow;    // row number
typedef int Ccol;    // column number
typedef int Cdia;    // diagonal item (row == column)
```

The translation from FORTRAN to C++ is somewhat tricky for two reasons:

1. FORTRAN indices run from 1..N while C++ indices run from 0..N-1
2. FORTRAN matrices are stored in column-major order (i.e. adjacent items in memory belong to the same column) while C++ matrices are stored in row-major order (i.e. adjacent items in memory belong to the same row).

Inverting an upper triangular matrix R

Suppose **R = op₁ • op₂ • ... • op_n • ID** where every **op_k** adds a multiple of some row of **R** to some other row of **R**. Then, as we learn in school, **R⁻¹ = ID • op₁ • op₂ • ... • op_n**. In other words, if the operations **op₁**, **op₂**, ..., **op_n** turn a matrix **R** into the identity matrix, then the same operations (in the same order) turn the the identity matrix into the inverse **R⁻¹** of **R**.

The operations that turn an upper triangular matrix **R** into the identity matrix **ID** are fairly simple:

1. **R** is upper triangular. Therefore the last row of **R**, say row **n**, has only one nonzero element **r_{nn}** on its diagonal. For every row above that last row, say row **k** (i.e. with $k < n$), do:
 1. Let **r_{kn}** be the last element in row **k**. Subtract a multiple of the last row from row **k** so that **r_{kn}** becomes **0**.
 2. That multiple is **r_{kn} ÷ r_{nn}**.
 3. Since all other elements of the last row are **0**, this only sets **r_{kn}** to **0** but does not change any other element of row **k**.
 4. After repeating this step for every $k < n$ was the entire last column of the matrix **R** is **0**, except for the element **r_{nn}** on the diagonal.
2. Repeat the same step for the second-last, third-last, ... first column of the matrix. In every iteration one column of **R** is set to 0 above the diagonal. After the last **k=1** the matrix is diagonal (i.e. all elements above and below the diagonal are **0**).
3. Finally, subtract a multiple of every row from itself so that the diagonal item becomes **1.0**.
 1. That multiple for diagonal element **r_{kk}** of row **k** is $(1 - r_{kk}) \div r_{kk}$.
 2. After repeating that step for every $k \leq n$ is **R** the unit matrix.
4. Notice that every operation above subtracts a multiple of some row from some other (or the same) row.

The algorithm above defines the sequence **op₁ ◦ op₂ ◦ ... ◦ op_n** above, where **n** is the number of columns in **R**. In practice (and in LAPack and consequently also in GNU APL) these operations are not performed on **R** first and then on the identity matrix afterwards but rather in parallel:

1. Start with a matrix **R, AUG** where **AUG ← (pR) ↑ (1 ↑ pR) ◦. = 1 ↑ pR**. That is, **AUG** is initially the unit matrix with the same shape as **R**. The matrix **AUG** (or sometimes the matrix **R,AUG**) is commonly known as the *augmented matrix*.
2. The operations described above are then simultaneously applied to the left half (i.e. to **R**) and to the right half (i.e. to **AUG**) until the left half **R** of **R,AUG** has become the unit matrix. At that point, the right half **AUG** has become the desired inverse **R⁻¹** of **R**.
3. All this happens in functions **LA_pack::invert_UTM()** and **LA_pack::invert_QUTM()**. The term **QUTM** stands for **Quadratic Upper Triangular Matrix** and takes advantage of the fact that the inversion of a non-quadratic upper triangular $M \times N$ matrix is essentially the inversion of its upper $N \times N$ sub-matrix. The rows $N+1..M$ are all zero and become zero columns in the inverse.

QR factorization

Householder matrix

For the moment we consider only real matrices; the complex case is almost

identical, except that $\mathbb{Q}\mathbf{X}$ becomes $\mathbb{Q}+\mathbf{X}$ in the complex case. $\mathbb{Q}+\mathbf{X}$ is called the *conjugate transpose* of $\mathbf{X}$ which is the same as the "normal" transpose if $\mathbf{X}$ is real.

Let $\mathbf{B}$ be an arbitrary $M \times N$ matrix.

Let $\mathbf{V} \leftarrow \mathbf{1} \downarrow \mathbf{B}[:, \mathbf{1}]$, i.e. $\mathbf{V}$ is the first column of $\mathbf{B}$ except the diagonal item $\mathbf{B}[\mathbf{1}; \mathbf{1}]$.

Let $\mathbf{L} \leftarrow +/\mathbf{V} \times \mathbf{V}$, i.e. $\mathbf{L}$ is (the square of) the Euclidean length of $\mathbf{V}$.

The Householder matrix $\mathbf{H} = \mathbf{H}(\mathbf{B})$ for matrix $\mathbf{B}$ is then defined as:

$$\mathbf{H} = \mathbf{I} - \frac{2}{\mathbf{L}} \times \mathbf{V} \circ. \times \mathbf{V}.$$

That is:

$$\mathbf{H} = (\mathbf{H})_{ij} = \begin{cases} 1.0 - 2 \times \mathbf{V}[i] \times \mathbf{V}[i] & \text{if } i=j, \text{ and} \\ - 2 \times \mathbf{V}[i] \times \mathbf{V}[j] & \text{otherwise.} \end{cases}$$

It follows immediately that:

1. $\mathbf{H}$ is *Hermitian* (aka. *symmetric* if $\mathbf{H}$ is real). I.e. $\mathbf{H} = \mathbb{Q}+\mathbf{H}$,
2. $\mathbf{H}$ is *unitary* (aka. *orthogonal* if $\mathbf{H}$ is real). I.e. $\mathbf{H}^{-1} = \mathbb{Q}\mathbf{H}$, and
3. $\mathbf{H}$ is *involutary*. I.e. $\mathbf{H} = \mathbf{H}^{-1}$

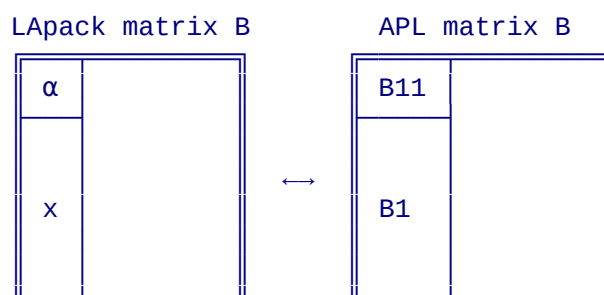
These really nice properties of $\mathbf{H}$:

1. make the computation of the transposition $\mathbb{Q}\mathbf{H}$ essentially a no-op, and
2. ensures that the composition $\mathbf{H} = \mathbf{H}_1 \circ \mathbf{H}_2 \circ \dots \circ \mathbf{H}_n$ of several Householder matrices is also Hermitian (complex case) resp. orthogonal (real case).

In the following let:

- $\mathbf{B}$ be an arbitrary real or complex matrix,
- vector $\mathbf{B1}$ be the first column of $\mathbf{B}$, and
- scalar $\mathbf{B11}$ be the first item of column vector $\mathbf{B1}$ (= the first item on the diagonal of $\mathbf{B}$).

In the LAPACK code is $\mathbf{B1}$ usually a variable named (lowercase) $\mathbf{x}$ and $\mathbf{B11}$ a scalar variable named (uppercase) $\mathbf{ALPHA}$ (or α in comments). That is, matrix $\mathbf{B}$ looks like this:



There is a subtle but noteworthy differences between the APL code discussed below and the Lapack FORTRAN or C++ code:

- In the original FORTRAN code of Lapack, the vector **x** *excludes* the diagonal element **α** and **α** is passed as a separate function argument or result.
- In the GNU APL C++ translation of that code, the vector **x** also *excludes* the diagonal element **α**. However, **α** is not passed as a separate function argument like in FORTRAN. Instead C++ takes advantage of the fact that our matrices are stored in FORTRAN order which makes **α** $\leftrightarrow$ **x**[-1] and therefore a function that already knows **x** can easily recover **α** from **x**.
- In the APL code below, the vector **B1** (which corresponds to **x**) *includes* the scalar **B11** (which corresponds to **α**).

Instead of a strict mathematical proof we use APL as a tool of thought.

Let **V** be a real or complex vector. The following APL function named **Householder** simply implements the definition of a Householder matrix given above:

```

▽H←Householder V;I;L
⌈
⌈ return the Householder matrix (aka. elementary reflector) for vector V
⌈
I←I∘.=I←⌈pV      ⌈ I: the (pV)×(pV) identity matrix,
L←+÷/V×+V        ⌈ L: ||V||2 i.e. the square of the norm of vector V
H←I-(2÷L)×V∘.×+V ⌈ H: the Householder matrix for vector V
▽

```

Next we define a somewhat random matrix **B** that shall prove our claims by example (aka. *incomplete induction*):

```

MN←(M N)←4 3      ⌈ choose a matrix size (M≥N)
⌈RL←+÷/⌈TS ⋄ B←?(2, MN)⌈p18 ⌈ prepare for random B
B←+÷9 0J9×[1]B    ⌈ some complex M×N random matrix B
B11←↑B1←B[;1]     ⌈ vector B1 is column 1 of matrix B, and scalar
B11 its first item

```

With these prerequisites we can "prove" the following

Proposition: Let

```

L11←×B11          ⌈ B11 normalized to length 1 (= ±1 for real B11)
LB1←(+÷B1×+B1)×0.5 ⌈ scalar LB1 is ||B1|| i.e. the length of B1

H1←Householder V1←B1 + L11×M↑LB1 ⌈ a Householder matrix (from V1)
H2←Householder V2←B1 - L11×M↑LB1 ⌈ another Householder matrix (from V2)

```

Then the first columns of matrices **B•H1** and of **B•H2** are 0 below the top-left item **B11** of **B**.

Proofs (by APL examples **B**):

```

▽H←Householder V;I;L
⌈

```

```

    A return the Householder matrix (aka. elementary reflector) for vector V
    A
    I←I-I◦.=I-1pV      A I: the (pV)×(pV) identity matrix,
    L←+/(V×V)          A L: ||V||2 i.e. the square of the norm of vector V
    H←I-(2÷L)×V◦.×+V   A H: the Householder matrix for vector V
▽

LB1←(+/B1×+B1)*0.5     A scalar LB1 is ||B1|| i.e. the length of B1
L11←×B11               A B11 normalized to length 1 (= ±1 for real B11)
H1←Householder B1+L11×M↑LB1
H2←Householder B1-L11×M↑LB1

    H = Q+H              A H is Hermitian
1 1 1
1 1 1
1 1 1

    (Q+H) = Q+H          A H is unitary
1 1 1
1 1 1
1 1 1

    H = Q+H              A H is involutory
1 1 1
1 1 1
1 1 1

    B
27J117  54J72  18J27
81J117  18J117 126J144
117J81  117J108 9J54
135J27  117J18  117J90

    2FH1 ◦ B      A notice the zeros in first column
-61.12J264.83 -33.28J233.03 -8.67J203.23
.00J.00       -52.41J25.80  86.09J69.90
.00J.00       19.00J47.42  -54.44J1.31
.00J.00       6.71J1.84   39.43J65.25

    2FH2 ◦ B      A notice zeros in the first column
61.12J264.83 33.28J233.03 8.67J203.23
.00J.00     -19.79J30.51 73.02J12.81
.00J.00     29.70J16.77 -95.79J74.13
.00J.00     -8.43J59.27 -25.25J16.46

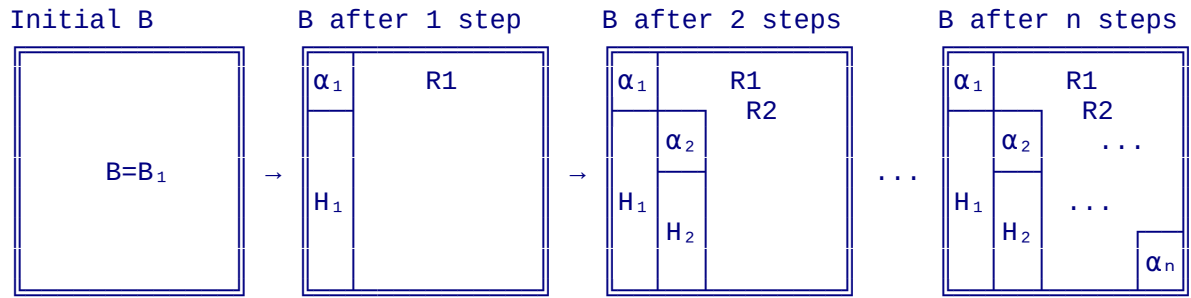
```

Repeated Application of Reflectors

Once we know how to set the first column of some arbitrary matrix **B** to **0** (below the diagonal) we can now turn the entire matrix into an upper triangular form:

1. Set **B₁ = B** and find a reflector **H₁** that sets the items below the first column of **B₁** to **0**.
2. Repeat that step for **B₂ = 1 1↓B₁**, **B₃ = 1 1↓B₂**, ... **B_n = 1 1↓B_{n-1}** which, in the end, computes a sequence of reflectors **H₁, H₂, ... H_n**. In this process:
 1. the matrices **B_k** become smaller and smaller,
 2. the reflectors **H_k** become shorter and shorter, and
 3. the upper triangular result matrix result becomes more and more complete.

After all, say **n**, steps matrix **B** looks like this:



For lack of space we show $\alpha_1 \dots \alpha_n$ on the diagonal above, but in reality the diagonal is $R_{11}, R_{22}, \dots R_{nn}$. The sequence $\alpha_1 \dots \alpha_n$ is stored in **tau** as will be explained below.

Now:

1. **R** is upper triangular (where the items below the diagonal are **0** by definition and therefore need not be stored in **B**)
2. $H \leftarrow H_1 \circ H_2 \circ \dots \circ H_n$ is unitary resp. orthogonal since it is the product of **n** unitary resp. orthogonal matrices H_k .

The iteration above takes place in function **LA_pack::laqp2()** which starts with matrix **B** and ends with **R** in the upper half (including the diagonal), **Q** in the lower half, and the diagonal factors $\alpha_1 \dots \alpha_n$ stored in variable **tau**.

At this point the lower half and **tau** are not the matrix **Q** itself but the reflectors that turn the identity matrix into **Q**. For this reason we usually refer to lower half as **HR** and not as **QR** in the GNU APL C++ code.

Numerical considerations

In mathematics we compute with exact numbers and would be finished at this point. On real computers we work with approximation of exact numbers which may cause problems with the accuracy of the result.

Let α and β be numbers. For the sake of explanation assume $\alpha > 0$ and $\beta > 0$. Due to rounding, do α and β only determine two ranges in which the exact α and β lie:

$$\begin{aligned} \alpha - e_1 &\leq \text{exact } \alpha \leq \alpha + e_1 \text{ and} \\ \beta - e_2 &\leq \text{exact } \beta \leq \beta + e_2 \end{aligned}$$

The (usually small) numbers e_1 resp. e_2 define the *absolute error* of α resp. β while the quotients $e_1 \div \alpha$ resp. $e_2 \div \beta$ define the *relative_error* of α resp. β .

Now, adding or subtracting the inequalities above gives:

$$\begin{array}{r} \alpha - e_1 \leq \text{exact } \alpha \leq \alpha + e_1 \\ \pm \quad \beta - e_2 \leq \text{exact } \beta \leq \beta + e_2 \\ \hline (\alpha \pm \beta) - (e_1 + e_2) \leq \text{exact } \alpha \pm \beta \leq (\alpha \pm \beta) + (e_1 + e_2) \end{array}$$

This means that the absolute error of the sum or difference ($\alpha \pm \beta$) is the sum of the absolute errors of α and β . This is no problem as long as $(\alpha \pm \beta)$ does not come too

close to **0**. If it does, however, then the absolute error remains small, but the relative error is now:

$$\frac{e_1 + e_2}{\alpha \pm \beta} \rightarrow \infty \quad \text{as } (\alpha \pm \beta) \rightarrow 0$$

That is, for **($\alpha - \beta$)** can the relative error become huge if **$\alpha \approx \beta$** . In LAPACK these issues are addressed in 3 ways:

1. If the input matrices are very large or very small (in terms of their norm) then they (and their result) are scaled up or down so that their coefficients land in a range that is not subject to major rounding error. See function **LA_pack::gelsy()** vs. **LA_pack::scaled_gelsy()**.
2. As we have seen above, for any matrix **B** there are actually two reflectors **H1** and **H2** each of which bring the first column off **B** below the diagonal to **0**. **LA_pack then chooses the one that produces the largest difference *V1 resp. V2 above.**
3. Some functions, in particular **LA_pack::larfg()** take additional measures to minimize the errors caused by too small arguments.
4. Column pivoting. instead of processing the columns in the order different from **B[;1]**, **B[;2]**, ... **B[N]** as to avoid small coefficients in the subsequent matrices.

The column pivoting works like this:

1. In every iteration above:
 1. Compute the norms of the columns,
 2. Find the column **k** with the largest norm (called "the pivot")
 3. If **k ≠ 1** (i.e. the first column does not have the largest norm already) then exchange columns **1** and **k** before proceeding.
 4. In the context of solving equations, the pivoting is essentially a renaming of the variables **x1, x2, ..., xn**. This renaming is later undone (see the end of **scaled_gelsy()**) before the result is returned to APL.

Memory Layout

At the time when FORTRAN was popular, memories were small. The LAPACK FORTRAN code addressed this in two ways:

Blocked execution

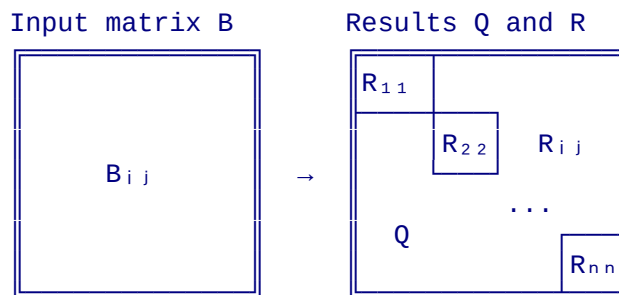
Blocked operation means that the processing of matrices that may not fit into the core memory of a computer were processed in chunks. Some FORTRAN functions therefore had a "blocked" version (for large matrices) and an "unblocked" version (for small matrices and for the chunks of large matrices).

These days memory is large and therefore blocking was removed entirely. Only the

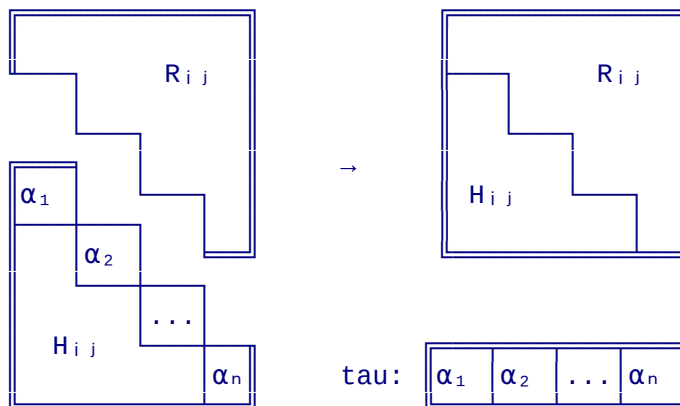
unblocked FORTRAN functions survived in the GNU APL code.

In-place computation

Another heavily used trick to reduce memory consumption is in-place computation. Instead of passing the argument of a function in one variable and storing the result in a another variable, the same variable was used for the largest input variable and the result. For example, function **laqp2()** computes the QR factorization of an arbitrary matrix **B** and the result is stored at the same memory location. However, the result **QR** is split in two pieces along the diagonal, taking advantage of the fact that **Q** is symmetric (so its upper half can be derived directly from its lower half, and that **R** is upper triangular (so its lower half is implicitly **0** and therefore not of interest):



Unfortunately in this scheme the matrices of **Q** and **R** now compete for the former diagonal of input **B**. This is solved by storing in the diagonal of **Q** in a separate vector (named **tau** in the LAPack code):



Another issue to understand LAPack is that functions **laqp2()** and friends do **not** return the matrices **Q** and **R** of the factorization $B = Q \cdot R$ directly. Instead of **Q** it returns a matrix **H** with the Householder reflectors for the different columns. The matrix **Q** can then be obtained by applying these reflectors to the $n \times n$ unit matrix **ID(n)**:

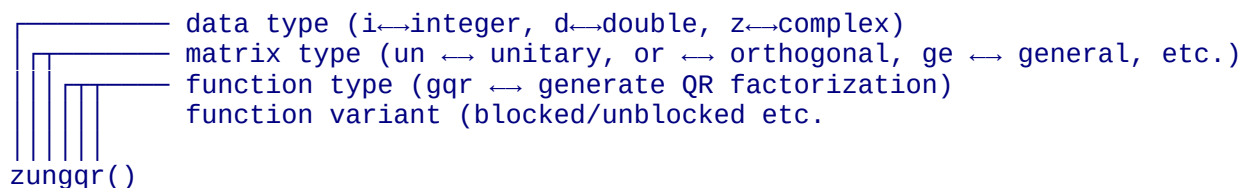
$$Q \leftrightarrow H \cdot \text{ID}(n)$$

This avoids one intermediate matrix multiplication. Instead of $Q \leftarrow H \cdot I$ followed by some other $Z \leftarrow Q \cdot R$, LAPack leaves **H** for the moment and later computes $Z \leftarrow H \cdot R$

directly. In addition LAPack avoids the computation of the householder matrices used in the APL examples above for demonstration purposes. Instead of full $k \times k$ reflector matrices it uses the reflector vectors, typically called **v**, and computes the elements **H_{ij}** of each reflector as needed. The lifetime of each **H_{ij}** is usually short, therefore it makes little sense to store it in a large matrix.

LApack function names

The names of LAPack functions look somewhat scary at first glance. For example the name of function **zungqr()** may not be too obvious. However these names follow a naming scheme that roughly looks as follows:



With this naming scheme function **zungqr()** for complex matrices becomes **dorgqr2()** for real matrices.

In FORTRAN every complex function (like **zungqr()**) has a real counterpart **dorgqr2()** for real matrices. In fact there are even more variants because there are different precisions (like 32-bit **float** and 64-bit **double** in C++) and each has its own FORTRAN functions. That made a lot of sense in the FORTRAN days where it made a considerable difference in performance.

A closer look at these functions revealed, however, that the difference between data types are actually rather small. The GNU APL version of these functions make the data type a template parameter so that every group of FORTRAN functions becomes a single template function in C++. In our example:

FORTRAN:

```
dorgqr  QR-factorize real matrix
zungqr  QR-factorize complex matrix
```

becomes C++

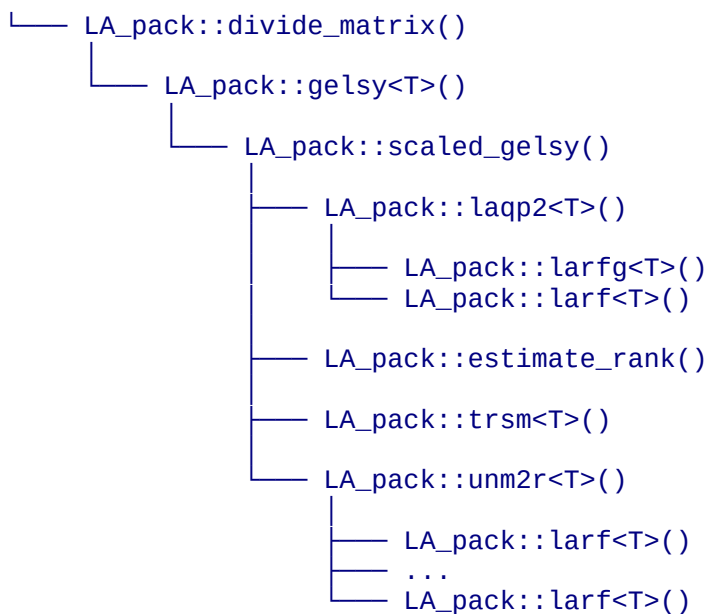
```
template<T> ungqr()
```

where the template argument **<T>** is either **<DD>** for real or **<ZZ>** for complex numbers. The use of C++ templates cuts the number of functions needed roughly in half,

Call Tree

With all thee in mind, **A_{EB}** aka. **Bif_F12_DOMINO::eval_AB** becomes simple. We look at its call tree, which is roughly this:

```
Bif_F12_DOMINO::eval_AB()
|
```



In this call tree:

- **eval_AB()** performs the APL argument checking (matching shapes of **A** and **B**) and figures if **A** or **B** contain complex items. It also treats scalars resp. vectors **A** and **B** as 1×1 resp. $1 \times N$ matrices
- **divide_matrix()** translates the ravels of APL matrices **A** and **B** into (transposed) FORTRAN matrices
- **gelsy<T>()** scales (and later unscales) FORTRAN matrices **A** and **B** with very small or very large items so that subsequent operations are safe in regard to rounding errors. In almost all real life cases **A** and **B** will not need any scaling.
- **scaled_gelsy()** computes $Z \leftarrow A \oslash B$ for the now properly scaled **A** and **B**
- **laqp2()** computes a QR factorization $B = Q \circ R$ of **B**. In the result:
 - **Q** is orthogonal (for real **A** and **B**) or Hermitian (for complex **A** or **B**). **Q** is easy to invert since $Q^{-1} \leftrightarrow Q + Q$, and
 - **R** is upper triangular (so that $R \circ X = C$ is simple to solve)
- In the QR factorization: for every column of **B**:
 - **larfg<T>()** generates one reflector (i.e. for one column of **B**)
 - **larf<T>()** applies that reflector to a sub-matrix of **B**
- **estimate_rank()** checks that the system $A = B \circ X$ of linear equations is not under-determined and raises a DOMAIN ERROR if it is. In contrast, over-determination (more equations than variables) is accepted and produces a least-square solution for $A = X = B \circ X$
- **trsm<T>()** computes R^{-1} for the upper triangular factor **R** of $B = Q \circ R$ that was computed above
- **unm2r<T>()** applies R^{-1} to **A**

with axis: QR factorization

The first non-trivial thing that happens in monadic $\mathbb{Q}\mathbb{B}$ and in dyadic $\mathbf{A}\mathbb{Q}\mathbb{B}$ is a *QR-factorization* of matrix **B**. A QR factorization of a matrix **B** is a pair **(Q R)** of matrices with the following properties:

1. **Q** is orthogonal (resp. Hermitian if **B** is complex),
2. **R** is upper triangular, and
3. $\mathbf{B} \equiv \mathbf{Q} \circ \mathbf{R}$

The same matrix **B** may have different QR factorization with the above properties. And different factorization algorithms may produce different pairs **(Q R)** for the same matrix **B**.

Often the orthogonal **Q** and/or the upper triangular **R** of a factorization $\mathbf{B} = \mathbf{Q} \circ \mathbf{R}$ is more valuable than the final result $\mathbf{B}^{-1} = \mathbf{R}^{-1} \circ \mathbf{Q}^{-1}$ of $\mathbb{Q}\mathbb{B}$ or $\mathbf{A}\mathbb{Q}\mathbb{B}$. For this reason GNU APL provides $\mathbb{Q}[\mathbf{X}]$ as to obtain the matrices **Q** and **R** without applying them (i.e. in order to solve $\mathbf{A} \circ \mathbf{X} = \mathbf{B}$). As a matter of convenience for the programmer, GNU APL not only computes the pair **(Q R)**, but a triple **(Q, R, Rinv)** where $\mathbf{Rinv} \equiv \mathbb{Q}\mathbf{R}$. One could, of course, compute **Rinv** from **R** in APL with $\mathbb{Q}$. But **Rinv** is anyhow computed in the QR factorization of **B** and therefore it is more efficient to return it together with **R** rather than computing it after **R** (which would QR-factorize **B** one more time).

GNU APL has implemented two different algorithms for the QR-factorization of a matrix **B**, which produce slightly different results.

1. The first algorithm is based on a factorization algorithm published by **Garry Helzer** in **APL Quote Quad, Volume 9, Issue 3, 1979**, and
2. the second algorithm is, like monadic and dyadic $\mathbb{Q}$, based on LAPack function **laqp2()** but with column pivoting disabled.

The algorithm to be used is selected with axis argument **X** as follows:

1. Integer scalar 1 selects Garry Helzer's algorithm,
2. Integer scalar 2 selects the laqp2() algorithm, and
3. (obsolete) a small real scalar like $\square\text{CT}$ also selects Garry Helzer's algorithm. This is for backward compatibility with older GNU APL versions (that always used Garry Helzer's algorithm).

For quadratic matrices **B**, i.e. for $\rho\mathbf{B} \leftrightarrow \mathbf{N} \mathbf{N}$, are the results of the two algorithms, apart from rounding errors, identical.

For non-quadratic matrices **B** with shapes $\rho\mathbf{B} \leftrightarrow \mathbf{M} \mathbf{N}$ with $M > N$, the results differ as follows:

	Helzer	laqp2()	Notes
axis X	1	2	algorithm selector
$\rho\mathbf{B}$	$\mathbf{M} \mathbf{N}$		$M \geq N$ (else LENGTH ERROR)

	Helzer	laqp2()	Notes
ρQ	M M	M N	rows N+1 ... M are non-zero
ρR	M N	N N	rows N+1 ... M are padded with 0
ρRinv	N M	N N	columns N+1 ... M are padded with 0
ρQ∘R		ρB	since $B \equiv Q \circ R$
N N ↑ R		Same	
N N ↑ Rinv		Same	
(Q∘Q)∘Q		ID N	$ID\ N \leftrightarrow (\iota N) \circ . = (\iota N)$
Q∘(Q∘Q)	ID N	≠ ID N	
Rinv∘R		ID N	
R∘Rinv	M M ↑ ID N	ID N	

For bread and butter applications is Helzer probably the alternative that is simpler to use. Lapack puts quite some effort into performance optimizations and numerical border cases. Therefore the Lapack alternative might be better suited for large matrices or when precision should become an issue.

Usage of $\square$ CR

All $\square$ -functions use $\square$ CT although in different ways.

$\square$ CT in the Helzer Algorithm

To quote Garry Helzer:

For this reason, whenever it is necessary to check an array for zeros below, the check is always made relative to another array and with a specified tolerance.

In GNU APL this comparison of small items with exact 0.0 can be controlled with $\square$ CT. The original Helzer algorithm used two APL functions **CPR** (for compare) and **TOL** (tolerance) to check for zeros:

```
0 ^.= (I+V) CPR L TOL DCT
```

which have been inlined in GNU APL, see **Bif_F12_DOMINO::significant()**.



the Helzer Algorithm in GNU APL, i.e. **⌈[1]B**, does not use any of the Lapack functions.

□CT in the LAPack functions

The LAPack based functions in GNU APL, i.e. $A\begin{smallmatrix} \square \\ \square \end{smallmatrix}B$, $\begin{smallmatrix} \square \\ \square \end{smallmatrix}B$, and $\begin{smallmatrix} \square \\ \square \end{smallmatrix}[2]B$, use the C++ function `LAPack::estimate_rank()` to compute the effective rank of the right argument **B**. Matrices with fewer rows than columns are *under-specified* and always yield a RANK ERROR. However a matrix can be under-specified even if the number of rows is sufficiently large. This is the case if the rows are not linearly independent. Put differently, a large number of rows alone does not guarantee sufficient information because a linear dependent row is as good as no row.

Function `estimate_rank()` has no corresponding FORTRAN function but was a somewhat lengthy section of `LAPack::gelsy()` that was factored out into `estimate_rank()` in order to improve the readability of the C++ code.

Function `estimate_rank()` iteratively computes two values **smin** (an upper bound for the smallest singular value of **B**) and **smax** (a lower bound for the largest singular value of **B**), and considers matrix **B** as rank-deficient if:

$$\text{smax} * \square\text{CT} > \text{smin}$$

If that happens then GNU APL reports a **RANK ERROR**. It means that the matrix **B** (i.e. of $A\begin{smallmatrix} \square \\ \square \end{smallmatrix}B$, $\begin{smallmatrix} \square \\ \square \end{smallmatrix}B$, or $\begin{smallmatrix} \square \\ \square \end{smallmatrix}[2]B$) is either truly rank-deficient, or at least close to a rank-deficient matrix. Increasing $\square\text{CT}$ may or may not solve that problem. Unlike "normal" RANK ERRORS, the meaning of this type of RANK ERROR is that, even though the shape of the matrix **B** is correct in the APL sense, the rows of **B** are at least close to being linearly dependent. In such cases the precision of the result is rather questionable and the RANK ERROR indicates that problem rather than continuing with the computation and producing an incorrect result.